```
    def android_v1
      # ...
    end
  end
```

Like `context`, Hotwire Native iOS is aware of the `view_controller` property, too. The framework exposes it when the user taps a new link via the `NavigatorDelegate`. As a quick refresher from Chapter 1, *Build Your First Hotwire Native Apps*, *delegates* are a design pattern in iOS. They provide a way for objects to act on behalf of others to handle specific events, like when a link is tapped.

Our `SceneDelegate` is the one creating the Navigators via `HotwireTabBarController`, so that's a great place to implement `NavigatorDelgate`. At the bottom of `SceneDelegate.swift`, add an extension that implements the `NavigatorDelegate` protocol and implements this function.

ch05_06/ios/App/Delegates/SceneDelegate.swift

```
import HotwireNative
import UIKit

let baseURL = URL(string: "http://localhost:3000")!

class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    // ...
}

» extension SceneDelegate: NavigatorDelegate {
»     func handle(
»         proposal: VisitProposal,
»         from navigator: Navigator
»     ) -> ProposalResult {
»     }
» }
```

`handle(proposal:)` is called every time the user taps a link. It gives us an opportunity to customize what type of screen is rendered. To do that, the function requires us to return a `ProposalResult`.